

Prompts utilizados en el TP

1. Actúa como un programador Junior en Go. Diseña un sistema de alta disponibilidad para la aprobación y análisis de fraude de transacciones financieras concurrentes. Una función debe cargar un lote secuencial de exactamente 50 transacciones enviándolas a un canal sin buffer llamado `transacciones_entrantes`. Instanciá exactamente 5 goroutines de validación (`workers`) en paralelo que consuman de ahí, tardando de forma aleatoria entre 100ms y 400ms por operación. Los `workers` deben enviar el string 'Transacción X aprobada por el Nodo Y' a un canal llamado `registro_contable`. El `main` debe imprimir los resultados y finalizar de forma limpia usando `sync.WaitGroup` y `close()` sin usar bloqueos mágicos de tiempo como `time.Sleep`.
2. ¿Sufre el programa del error crítico de lanzar un `Panic` por escribir en un canal cerrado?
3. Vuelve a analizar el código que me diste anteriormente y ahora refactorízalo pensando en que las redes bancarias fallan agregando un `select` con `time.After` dentro del bloque de los `workers`. Si la consulta simulada tarda más de 250ms, el `worker` debe abortar la operación, enviar el mensaje "Transacción X rechazada por Timeout de Red" al canal de resultados y continuar con la siguiente transacción.
4. Revisa el código nuevamente, en el primer prompt utilizado dice específicamente "sin usar bloqueos mágicos de tiempo como `time.Sleep`" corregí esto sin cambiar partes que ya funcionaban.
5. Te enviaré un código en GO y respondeme estas preguntas: 1) por qué falla el programa y donde ocurre un bloqueo? 2) si eliminamos el 2 en la creación del canal, el programa fallaría porque el `log1` se bloquea ya que nadie recibe el mensaje?

ACLARACIÓN: La IA utilizada para el punto 3 que recibió el prompt fue Claude de anthropic (específicamente Sonnet 4.6), adjuntamos 3 códigos ("`codigo_inicial`", "`codigo_refactorizado`", y "`codigo_sin_sleep`"). El primero corresponde a la parte 3, y los dos últimos a la parte 4 en la consigna que pide refactorizar. En el primer resultado, la IA ignoró el pedido de no utilizar `time.sleep`, en el segundo resultado se refactorizó el código pensando en que las redes bancarias puedan fallar, y en el tercer código recibido, intercambió `time.sleep` por un `select` con un `time.after` ya que hasta el momento utilizó (si bien le aclaramos que no utilice el `time.sleep`) el `time.sleep`, además modificamos manualmente caracteres que no compilaban ya que no pertenecían a utf